

E: Spacebar Tokenizer

El tokenizador de la barra espaciadora

Nombre del archivo fuente (*Source file name*): `spacebar.c`, `spacebar.cpp`, `spacebar.java` o `spacebar.py`

Autor (*Author*): Juan Corena

Spacebar Tokenizer es una startup que desarrolla un tokenizador de última generación. Su diseño único ayuda a los programadores más rápidos del mundo a no perder tiempo presionando la barra espaciadora mientras escriben.

Para lograr este objetivo, el tokenizador usa un sistema de puntuación basado en un modelo de lenguaje grande. Este modelo asigna una puntuación entera a cada *token* válido del lenguaje y una puntuación de 0 a cualquier token que no sea reconocido. Al tokenizar una oración, el tokenizador suma las puntuaciones de cada token según el modelo. Una *tokenización óptima* (*optimal tokenization*) es aquella que produce la puntuación total más alta entre todas las formas posibles de tokenizar la oración dada.

Por ejemplo: `ilovespacebartokenizer` podría tokenizarse de varias maneras, entre ellas, aunque no únicamente: «`i love space bar tokenizer`», «`i love spacebar tokenizer`» e «`ilovespace bartokenizer`». Si el modelo de lenguaje asigna una puntuación de 5 al token `spacebar`, 2 al token `bar` y 3 al token `tokenizer`, entonces la segunda tokenización produce la puntuación más alta entre las 3 tokenizaciones del ejemplo, con una puntuación total de 8. Observa que puede haber casos en los que varias tokenizaciones den como resultado la misma puntuación máxima.

La siguiente tabla muestra las puntuaciones de las tokenizaciones del ejemplo. Los tokens que contribuyen a la puntuación final están en negrita:

Tokenización (<i>Tokenization</i>)	Puntuación (<i>Score</i>)
i love space bar tokenizer	5
i love spacebar tokenizer	8
ilovespace bartokenizer	0

Tu tarea es crear un programa que imprima la puntuación que debería dar un tokenizador cuando se comporta de manera óptima, como se describió.

Entrada (Input)

La entrada consiste en varios casos de prueba. Cada caso de prueba consiste en un modelo de lenguaje que da la puntuación de cada token y una serie de oraciones que deben tokenizarse de manera óptima según el modelo de lenguaje.

Cada caso de prueba comienza con una línea que contiene dos enteros $1 \leq m \leq 1\,000$ y $1 \leq s \leq 100$ separados por un solo espacio. Cada una de las siguientes m líneas contiene, separados por un espacio, un único token formado por letras minúsculas del alfabeto inglés y un número entero positivo que indica la puntuación que ese token aporta a la puntuación total de la tokenización. La longitud de cada token es al menos 1 y a lo sumo 100 caracteres; se puede suponer que el token es único. Luego siguen s líneas, cada una con una secuencia a_i , para $1 \leq i \leq s$, de caracteres en minúscula del alfabeto inglés que se debe tokenizar, donde $1 \leq |a_i| \leq 1\,500$.

La entrada termina con dos 0 separados por un espacio.

La entrada debe leerse de la entrada estándar.

Salida (Output)

Para cada caso de prueba y cada a_i , imprime una sola línea con la puntuación de la tokenización óptima de a_i según el modelo correspondiente.

La salida debe escribirse en la salida estándar.

Ejemplo de entrada (Sample Input)	Ejemplo de salida (Sample Output)
7 2 space 5 bar 7 spacebar 13 tokenizer 5 i 2 love 4 ipo 8 ilovespacebartokenizer theonlytokenizeripo 0 0	24 13